

FOR SOFTWARE ENGINEERS: A METHOD, NOT A TOOL COMPARISON

The Bug You Cannot Reproduce

Do not make one agent both the author and the judge

Green 40 of 40 on your laptop. Red 1 in 12 in CI. Three days of retrying pipelines for a four line fix.

The test passes on your machine every time and fails in the pipeline about once in twelve. You have rerun the build more times than you want to admit.

There is one step in the middle where the obvious move is the wrong move, and it decides whether you fixed the bug or moved it.

Any statement about what a particular tool can do is dated September 2026 and will age badly. That is exactly why nothing on this page depends on one: the method is about which ROLE a tool is playing, which does not change when the feature list does.

In scope

- Why asking one assistant to check its own theory tells you nothing
- Asking for the SEARCH SPACE rather than the answer
- Handing the two hundred runs to the thing that has a terminal
- Sending the fix to the second opinion COLD, and why that word matters
- When this whole method is not worth doing

Out of scope

- Which tool is better. The capabilities overlap and overlap more every month
- Any claim that a second opinion finds bugs a person would not
- Prompt templates to paste without reading. The shape is the point
- Anything about how this channel is produced, which is not the subject

The method survives the tools. Read it as two ROLES that must not be held by the same participant, and it keeps working when the feature lists change.

01 The method, and what each step is protecting

1. Split the roles	One proposes. One executes and verifies. Never the same one
2. Ask for hypotheses	Ranked, each with the cheapest test that disproves it
3. Say: do not give me the fix	A fix stops the thinking. A search space starts it
4. Hand over the grind	Two hundred runs, failures SAVED rather than summarised
5. Send the fix cold	The diff, never the reasoning that produced it
6. Read the diff yourself	You are the only participant accountable on Monday

Every step exists to stop the second opinion being contaminated by the first. That is the single idea the whole page is made of.



03 The instinct that fails

You paste the failure into one assistant. It gives you a confident theory. You ask it to check its own theory, and it agrees with itself.

Of course it agrees. You asked the author to be the judge.

State it plainly, because the rest of this page is this one sentence applied: do not make one agent both the author and the judge. Give different jobs to things with different context and different failure modes.

A reviewer that produced the work under review shares every assumption that produced it, so its agreement carries no independent information.

First principles, not vendor documentation

04 It is not about capability, it is about role

The myth is that one of these has the internet and the other has your terminal. That is out of date. At the time of writing, in September 2026, both sides of this pairing can search and both can run commands in a repository.

So the split that matters is not capability. It is ROLE. One proposes, the other executes and verifies, and then they swap, so the thing that wrote the fix is never the thing that approves it.

Writing the method around capability would make it wrong within months. Writing it around role makes it survive the next release of everything.

Capability overlap between the two tools was checked in September 2026 and is expected to increase, so no step here depends on one of them holding a capability the other lacks.

Checked September 2026, and capability claims expire

05 Ask for the search space, not the answer

You are not asking for the answer. You are asking for the SEARCH SPACE, and that distinction is most of the skill.

Ask for ranked hypotheses, and for each one the CHEAPEST TEST THAT WOULD DISPROVE IT, and say explicitly: do not give me the fix.

Ask for the fix and you get one confident answer and you stop thinking. Ask for five ranked hypotheses with disproof steps and you get a plan you can run.

A ranked list with a disproof step per item is executable; a single confident answer is not, and it ends the search.

Engineering practice, not vendor documentation

06 Hand over the part you have been avoiding

Somebody has to run that test two hundred times, capture every failure, and find what the failing runs have in common. That is the job you have been putting off for three days, and it is exactly what a tool with a terminal is for.

Fix the number of runs in the prompt. Demand the failures be SAVED, not summarised. End with one question: which hypothesis survived.

You are not asking for an opinion. You are asking for evidence, from the participant that can actually produce it.

A summary of failures discards the field that distinguishes them, which is usually the one that identifies the cause.

Engineering practice, not vendor documentation

Two hundred runs with every failure retained is the evidence the three days of reruns never produced.

Run on the machine that built this sheet, not quoted

07 The culprit, and the insult in it

The fix is four lines. Three days of retrying pipelines for four lines.

The health check was asking whether the process was up. What it needed to ask was whether the schema was at the expected version. Same wait, different question.

This was never a hard problem. It was an expensive problem, and the expense was entirely in the two hundred runs nobody wanted to sit through.

A readiness check that tests liveness rather than the state the test depends on will pass before the system is ready, intermittently.

First principles, not vendor documentation



08 Send it cold, and let them disagree

You have a fix. Four lines, two hundred runs, two hundred green. The obvious move is to ask the tool that did all that work whether the fix is any good. Do not. It watched itself reason its way here. It is invested, and it will tell you what it already believes in a very calm voice.

Send the artefact to the other one, and send it COLD: the diff, not the reasoning that produced it.

They disagree. The reviewer accepts the race is real and the fix addresses it, then points out the wait has no upper bound, so a migration that never completes turns a flaky test into a pipeline that hangs until the job times out.

That is a good catch, and it is a catch the first one was never going to make, because it was busy being pleased the tests went green. THE DISAGREEMENT IS THE PRODUCT.

A reviewer given the reasoning is being asked to check a conclusion it has already been argued into; a reviewer given only the artefact is being asked to form its own.

First principles, not vendor documentation

An unbounded wait converts an intermittent failure into a hang, which is a different and worse failure mode than the one being fixed.

Engineering practice, not vendor documentation

09 When not to bother

When the task is small, because moving artefacts between two windows for a one line change costs more than it saves.

When you cannot say what verifying would even mean. Then you do not have a review, you have a second opinion about a vibe.

And above all, when a test could answer the question. If a failing assertion settles it in nine seconds, do not pay anything to have an opinion. Where evidence is cheap, go and get the evidence.

A review with no stated criterion for passing is not a review.

First principles, not vendor documentation

Where a test can settle a claim more cheaply than a discussion can, the test is the correct instrument.

Engineering practice, not vendor documentation

The four rules, and the questions behind them

1. Different jobs, not different brands. Who proposes, and who verifies?
2. Is the reviewer seeing the artefact, or the argument that produced it?
3. Whenever a claim can be settled by running something, was it run?
4. Did I read the diff, given I am the only one accountable on Monday?
5. Did I ask for hypotheses, or did I ask for the answer?
6. Does each hypothesis come with the cheapest test that kills it?
7. Were the failures saved, or summarised?
8. Is the number of runs fixed in the prompt, or left open?
9. Does the fix introduce a wait, and does that wait have an upper bound?
10. Is this task big enough to be worth the two windows at all?

Everything else



Site

<https://code-with-sam-dev.github.io>



YouTube

<https://www.youtube.com/@CodewithSam-Dev>



LinkedIn

<https://www.linkedin.com/company/code-with-sam-dev>



x

<https://x.com/CodeWithSamDev>



TikTok

<https://www.tiktok.com/@codewithsamdev>



Instagram

<https://www.instagram.com/codewithsamdev>



Facebook

<https://www.facebook.com/codewithsamdev>



GitHub

<https://github.com/code-with-sam-dev>



Buy me a coffee

<https://buymeacoffee.com/codewithsam>



Buy me sadza

<https://buymesadza.com/codewithsam>

If this was useful, a like and a subscribe help more than you would think. And tell me in the comments which of the four your team actually does.

Study findings are attributed to their authors and were read at the primary source rather than from a summary. Product and company names appear only to identify what was studied. Nothing here implies endorsement by, or affiliation with, any of them.

Technical claims verified 2026-09-16 against the documentation named beside each one. Vendors change behaviour and defaults between versions, so check the version you are actually running before you rely on any of it.